

Testing de Software

Clase 1: Calidad | Validación y Verificación | Especificaciones y V&V | análisis estático y dinámico | Inspecciones | Testing | Procesos de desarrollo | Terminología | Clasificación

Valeria Bengolea

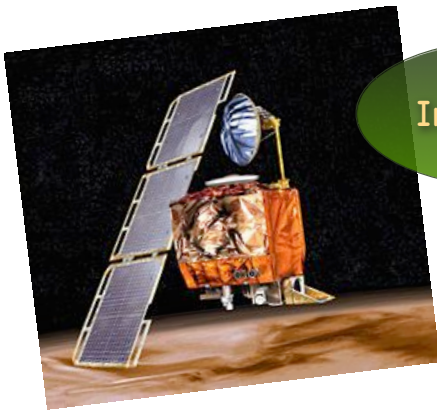
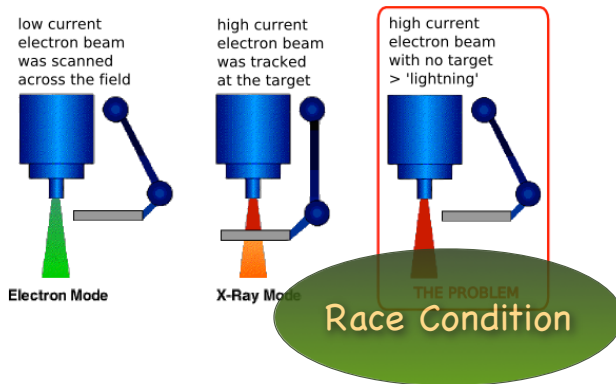
Estas transparencias están basadas en las desarrolladas por Ammann & Offutt como acompañamiento de su libro Introduction to Software Testing (2nd Edition) y por Manuel Núñez

Calidad en el Desarrollo de Software

- Uno de los objetivos principales en el desarrollo de software es conseguir productos de alta calidad.
- La calidad involucra confiabilidad, mantenibilidad, interoperabilidad, etc.
 - La **confiabilidad** es uno de los atributos de calidad más importantes.
- Una de las medidas de calidad mejor establecidas es la cantidad de defectos por líneas de código.
 - Mayor cantidad de defectos hace al software menos confiable (de menor calidad).

Desarrollar Software Confiable no es Trivial

Therac-25:



Mars Climate Orbiter:
Métrico vs Imperial



ariane 5:
64 bits fp
vs 16 bits int



Voto electrónico:
Integridad/Confidencialidad

Desarrollar Software Confiable no es Trivial

Microsoft Zune music players



Desarrollar Software Confiable no es Trivial

Microsoft Zune music players



```
public static int currentYear (int days) {  
    int year = 1980;  
    while (days > 365) {  
        if (isLeapYear(year)) {  
            if (days > 366) {  
                days -= 366;  
                year += 1;  
            }  
        } else {  
            days -= 365;  
            year += 1;  
        }  
    }  
    return year;  
}
```

Relevancia de la Confiabilidad del Software

Reporte de NIST “The economics Impacts of Inadequate Infrastructure for Software Testing” (2002)

Testing inadecuado cuesta a la economía de EEUU entre U\$S 22.000M Y 59.000M anuales

Mejores enfoques a QA pueden reducir este costo a la mitad

Grandes pérdidas debido a fallas de aplicaciones (en EEUU):

Servicios financieros: U\$S6.5M por hora

Tarjetas de crédito: U\$S2.4M por hora

**de Introduction to Software Testing (amman & Offutt 2016)*

Validación y Verificación

- Las tareas de validación y verificación ayudan a mostrar que el software cubre las expectativas para las cuales fue construido.
 - ayuda a garantizar calidad (en particular, confiabilidad).
- Esto no significa garantizar que el software será completamente libre de defectos.
 - Debe ser “lo suficientemente bueno” para el uso que se le pretende dar (y el uso determina el grado de confianza que se requiere del software)

El Problema de la Confiabilidad: Las Fallas

- Las fallas de software son **distintas** de las fallas mecánicas o eléctricas.
- En software, las fallas **no** son consecuencia del uso y el deterioro.
- Las fallas ocurren como consecuencia de defectos (o “bugs”) introducidos **durante** el desarrollo.
- Es decir: **la falla que causa el problema existe desde el comienzo**, sólo que se manifiesta tarde.

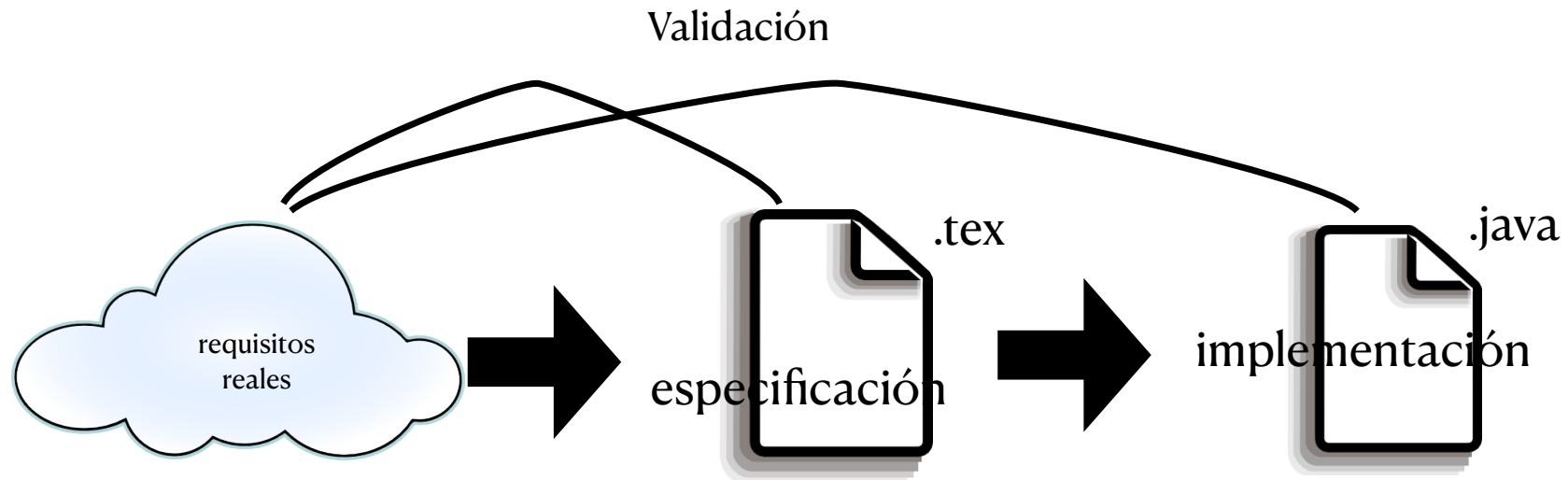
Validación vs. Verificación

- **Verificación:** el software debería realizar lo que su especificación indica:
 - ¿construimos el producto correctamente?
- **Validación:** El software debería hacer lo que el usuario requiere de él:
 - ¿construimos el producto correcto?

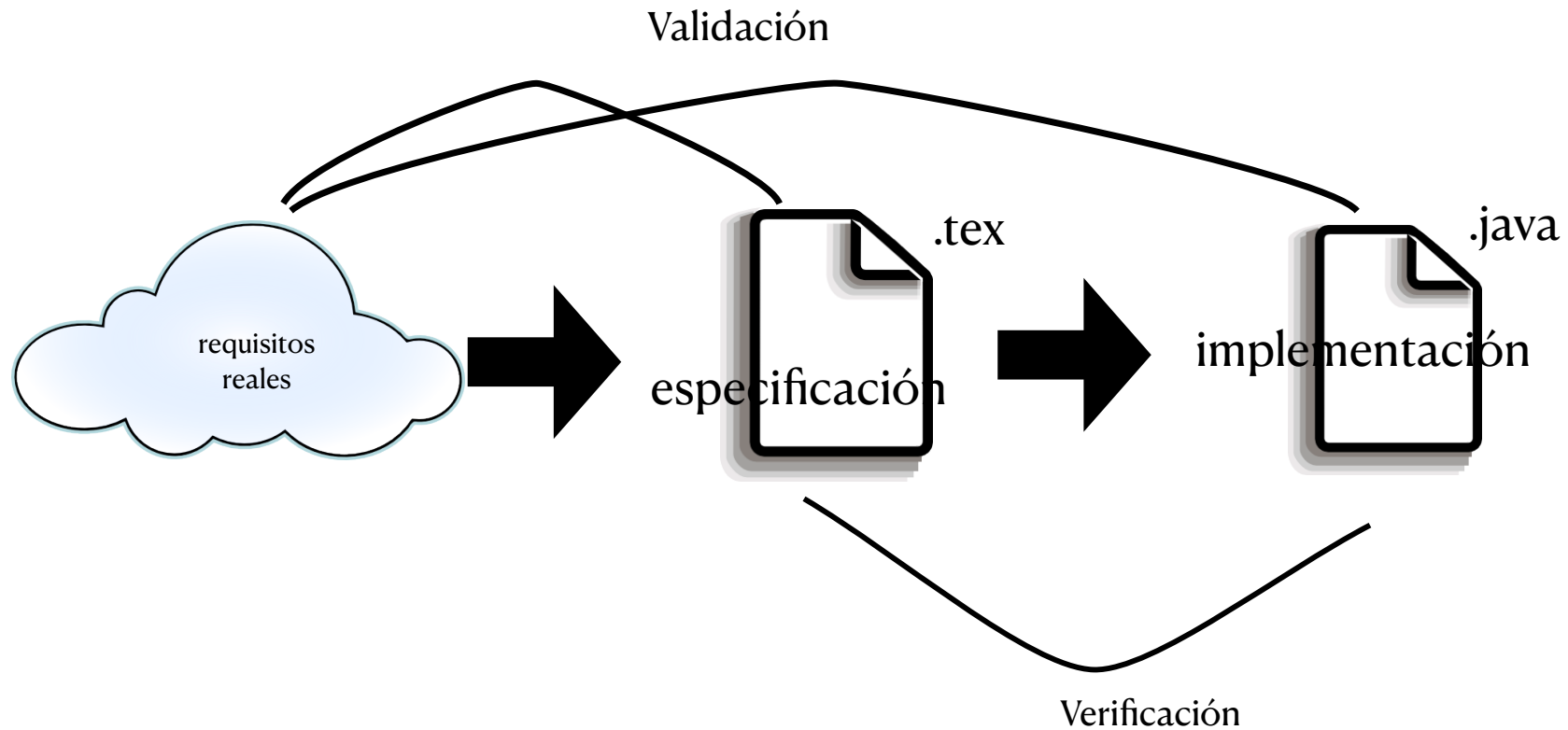
Validación vs. Verificación



Validación vs. Verificación



Validación vs. Verificación



Verificación de Software

Problema General

$$Sys \models Spec$$

Verificación de Software

Problema General

$$Sys \models Spec$$

Se comporta **Sys** de acuerdo a su especificación?

Verificación de Software

Problema General

Qué significa eso?

$\{pre\}$

$\models \Box \Diamond \alpha$

$\{post\}$

Verificación de Software

Problema General

Qué significa eso?

$\{pre\}$

Sys

$\models \Box \Diamond \alpha$

$\{post\}$

Verificación de Software

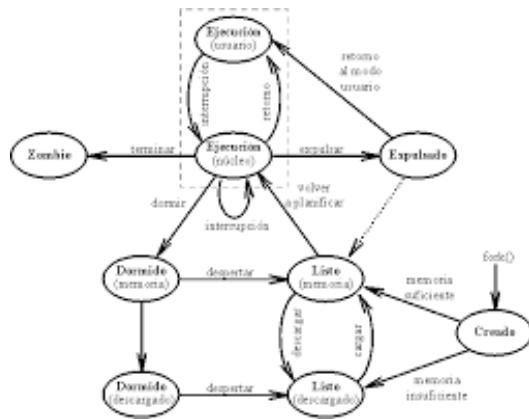
Problema General

Qué significa eso?

$\{pre\}$

Sys

$\{post\}$



$\models \Box \Diamond \alpha$

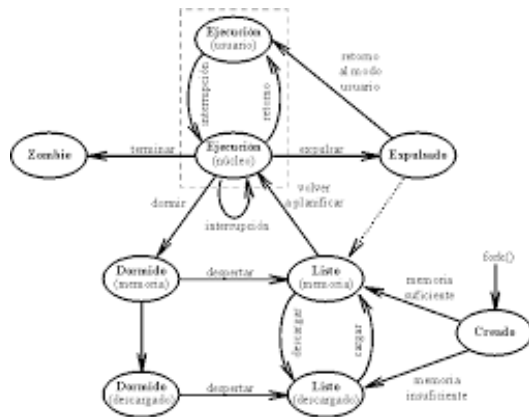
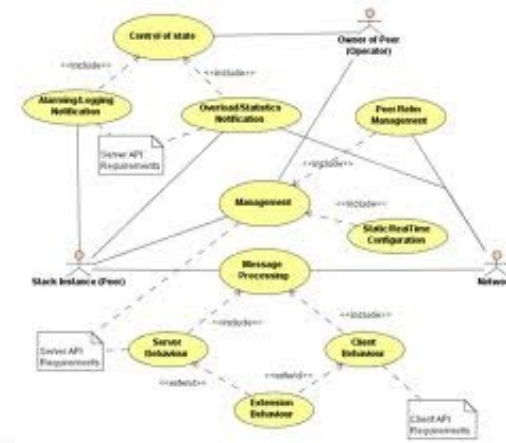
Verificación de Software

Problema General

Qué significa eso?

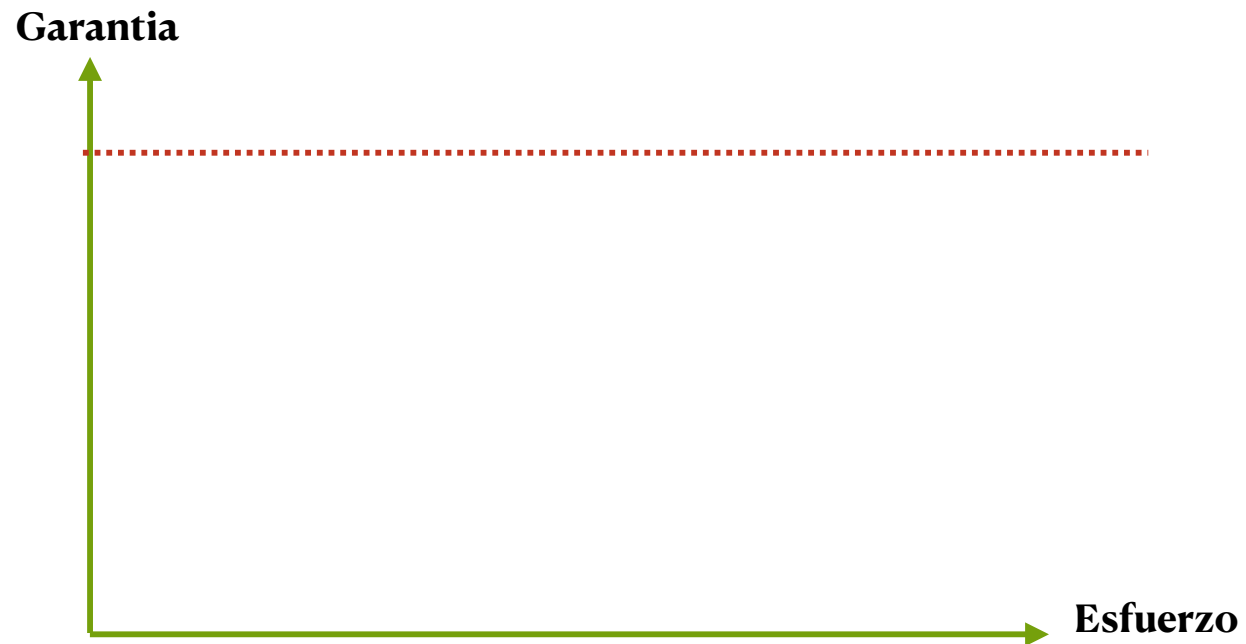
$$\{pre\}$$

Sys

 $\{post\}$ 
$$\models \Box \Diamond \alpha$$


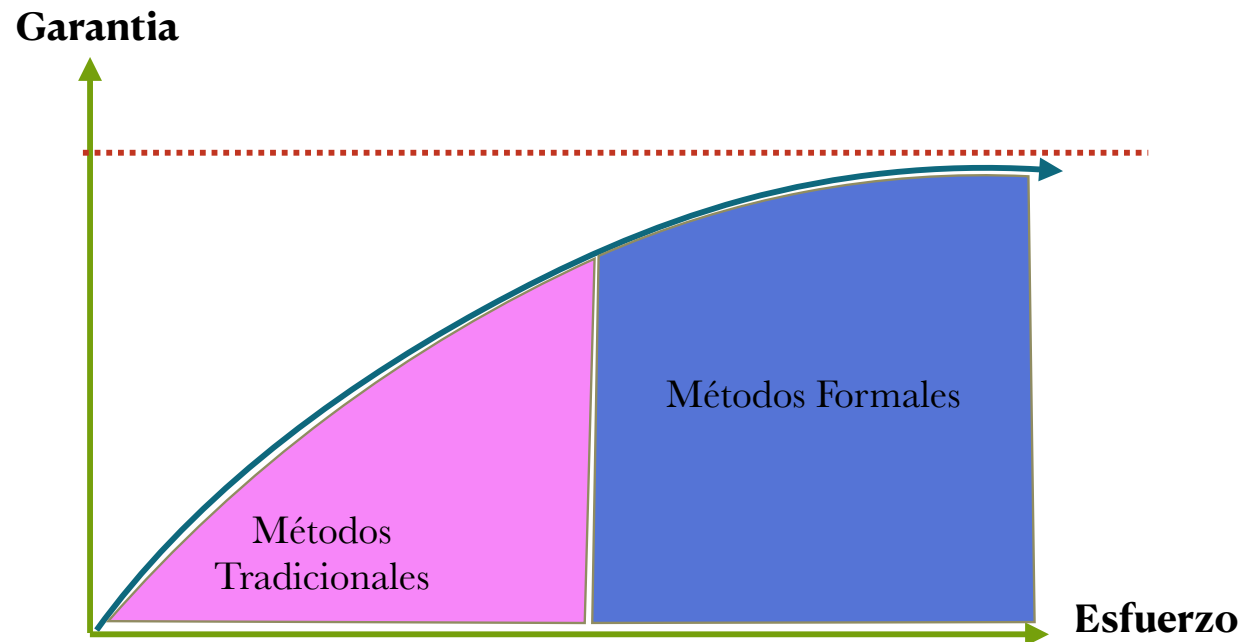
Enfoques

- **Tradicional**: No se cuenta con nada formal (excepto por el programa ejecutable).
- **Formales**: Verificación deductiva, Model Checking



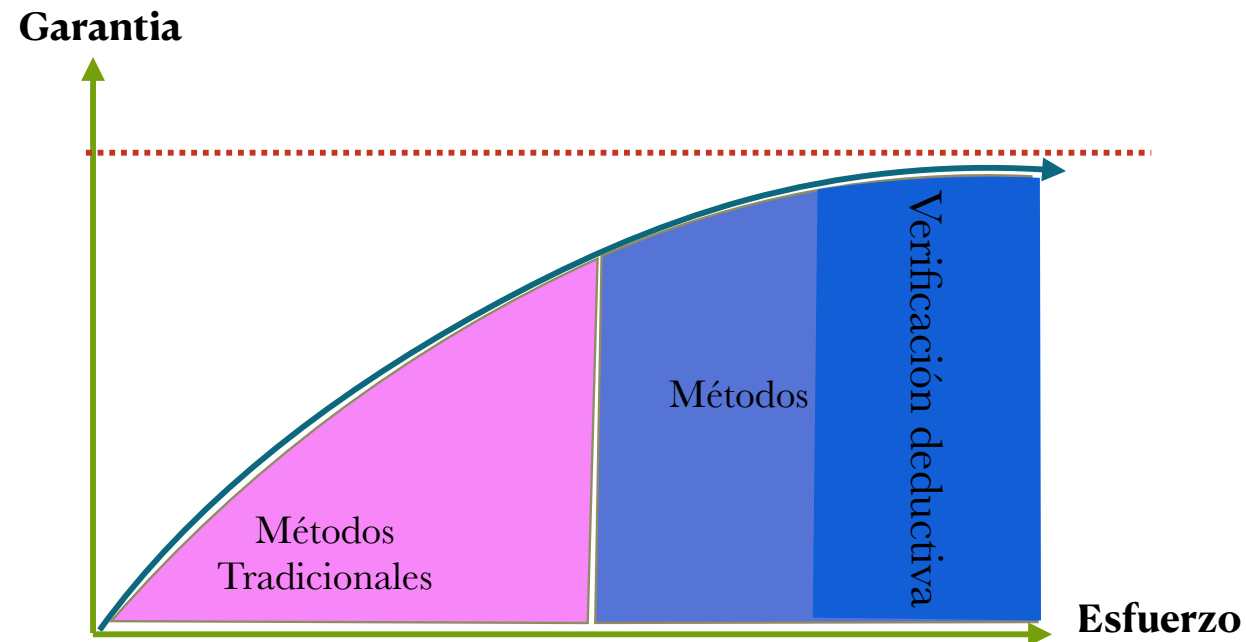
Enfoques

- **Tradicional**: No se cuenta con nada formal (excepto por el programa ejecutable).
- **Formales**: Verificación deductiva, Model Checking



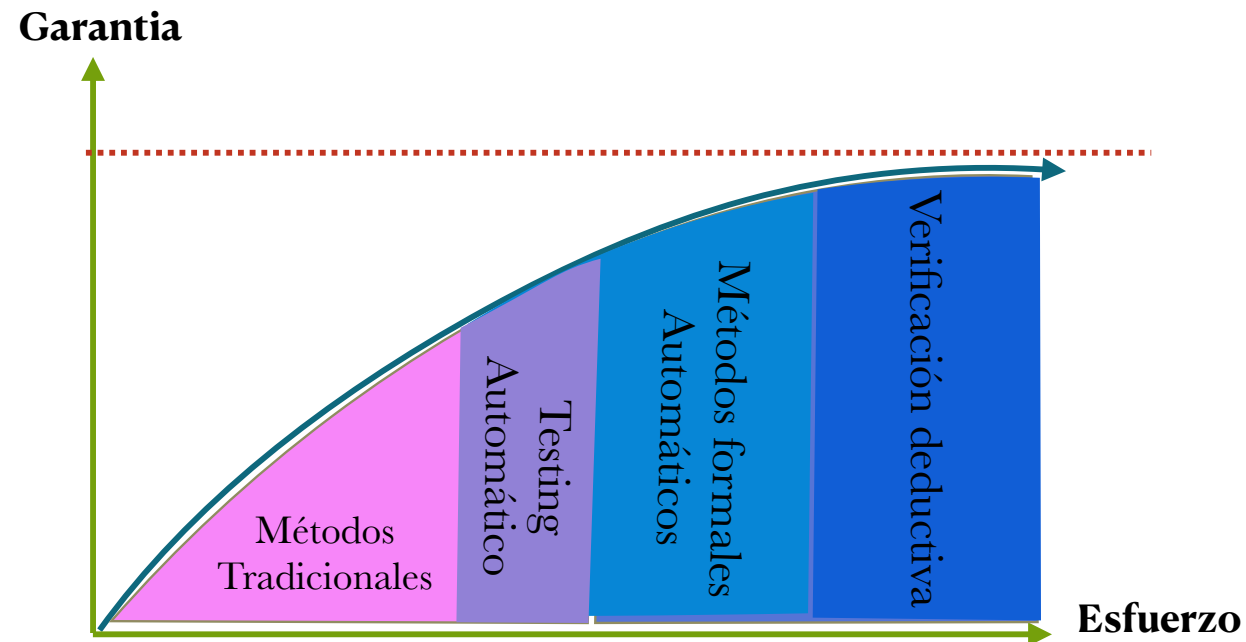
Enfoques

- **Tradicional**: No se cuenta con nada formal (excepto por el programa ejecutable).
- **Formales**: Verificación deductiva, Model Checking



Enfoques

- **Tradicional**: No se cuenta con nada formal (excepto por el programa ejecutable).
- **Formales**: Verificación deductiva, Model Checking



Especificaciones en V&V

Las especificaciones son fundamentales como mecanismo para poder razonar sobre la corrección de un programa, es decir, para las actividades de Validación y Verificación

La especificación de un programa es una descripción de qué es lo que se supone que el programa debe hacer.

Especificaciones en V&V

Existen muchas formas diferentes de expresar especificaciones de programas

una particularmente adecuada es la basada en aserciones de corrección

Aserciones de Corrección

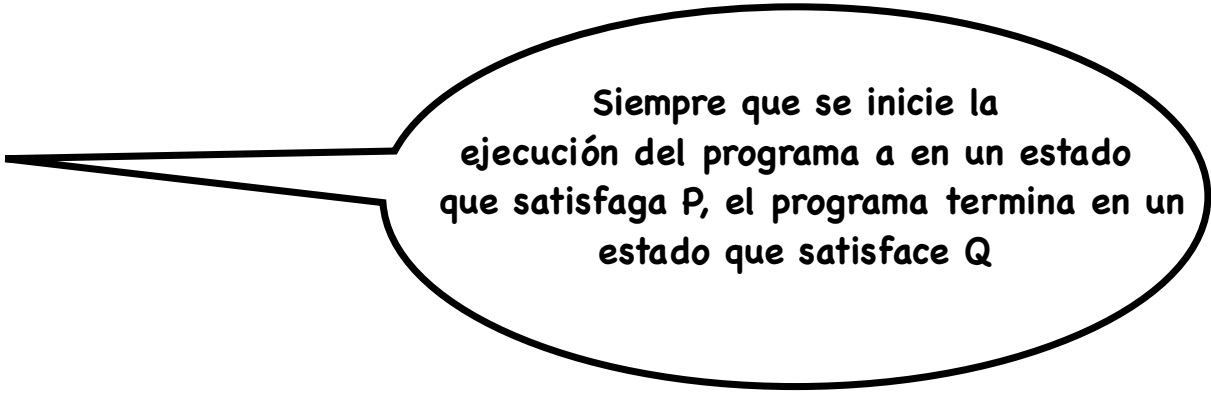
Las **aserciones de corrección de programas**, en particular para programas imperativos, suelen venir en el siguiente formato, de **pre- y post-condición**:

$$\{P\} \text{ a } \{Q\}$$

Aserciones de Corrección

Las **aserciones de corrección de programas**, en particular para programas imperativos, suelen venir en el siguiente formato, de **pre- y post-condición**:

$\{P\}$ a $\{Q\}$



Siempre que se inicie la ejecución del programa a en un estado que satisfaga P, el programa termina en un estado que satisface Q

Contratos: Derechos y Obligaciones

Las especificaciones de rutinas mediante **pre- y post-condiciones** establecen derechos y obligaciones para cliente y proveedor:

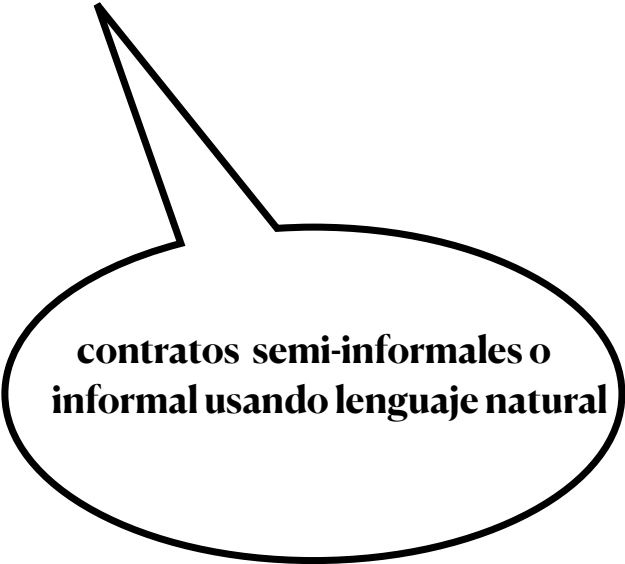
	Precondición	Postcondición
Cliente	DEBE asegurarse que se cumple (antes de la ejecución)	PUEDE suponer que se cumple (luego de la ejecución)
Proveedor	PUEDE suponer que se cumple (antes de la ejecución)	DEBE asegurarse que se cumple (luego de la ejecución)

Ejemplo

```
/**
@precondition. a != null, 0 <= i <= j < |a|
@postcondition. - a no cambia.
    - Si |a|=0, resultado = (0,0), sino resultado.fst() es el mínimo de a en el
    segmento [i..j] y resultado.snd() es el máximo de a en el segmento [i..j]
*/
public static IntPair minMaxarray(int[] a, int i, int j) {
    if (a.length == 0) {
        IntPair res = new IntPair(0,0);
        return res;
    }
    else {
        IntPair res_sub = new IntPair(a[i],a[i]);
        int k = i;
        while (k<=j) {
            if (a[k]<res_sub.fst()) {
                res_sub.changeFst(a[k]);
            }
            if (a[k]>res_sub.snd()) {
                res_sub.changeSnd(a[k]);
            }
            k++;
        }
        return res_sub;
    }
}
```

Ejemplo

```
/**
@precondition. a != null, 0 <= i <= j < |a|
@postcondition. - a no cambia.
    - Si |a|=0, resultado = (0,0), sino resultado.fst() es el mínimo de a en el
    segmento [i..j] y resultado.snd() es el máximo de a en el segmento [i..j]
*/
public static IntPair minMaxarray(int[] a, int i, int j) {
    if (a.length == 0) {
        IntPair res = new IntPair(0,0);
        return res;
    }
    else {
        IntPair res_sub = new IntPair(a[i],a[i]);
        int k = i;
        while (k<=j) {
            if (a[k]<res_sub.fst()) {
                res_sub.changeFst(a[k]);
            }
            if (a[k]>res_sub.snd()) {
                res_sub.changeSnd(a[k]);
            }
            k++;
        }
        return res_sub;
    }
}
```



contratos semi-informales o
informal usando lenguaje natural

Ejemplo: JML Spec

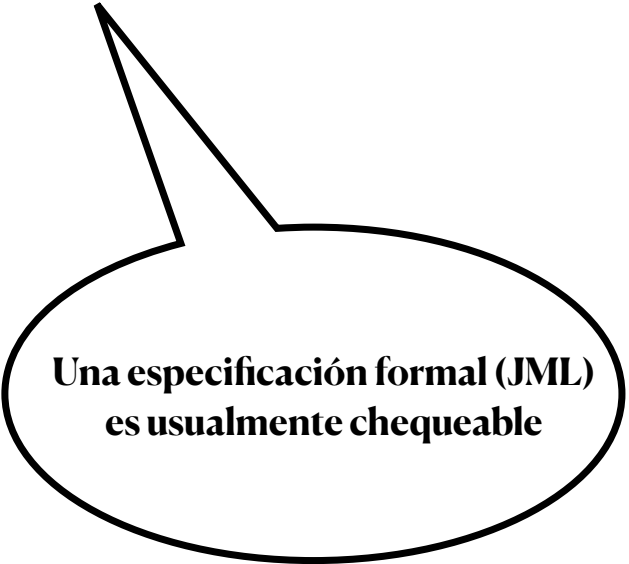
```
/*@ requires a!=null && i>=0 && j>=i && a.length > j;  
@ ensures (a == \old(a)) &&  
@      (a.length != 0 ==>(\forallall int k; i <= k && k <= j; a[k] >= \result.fst()  
      && a[k] <= \result.snd())) ) &&  
@      (a.length == 0 ==> \result.fst()==0 && \result.snd()==0);  
@*/
```

```
public static IntPair minMaxarray(int[] a, int i, int j) {  
    if (a.length ==0) {  
        IntPair res = new IntPair(0,0);  
        return res;  
    }  
    else {  
        IntPair res_sub = new IntPair(a[i],a[i]);  
        int k = i;  
        while (k<=j) {  
            if (a[k]<res_sub.fst()) {  
                res_sub.changeFst(a[k]);  
            }  
            if (a[k]>res_sub.snd()) {  
                res_sub.changeSnd(a[k]);  
            }  
            k++;  
        }  
        return res_sub;  
    }  
}
```

Ejemplo: JML Spec

```
/*@ requires a!=null && i>=0 && j>=i && a.length > j;  
@ ensures (a == \old(a)) &&  
@      (a.length != 0 ==> (\forall int k; i <= k && k <= j; a[k] >= \result.fst()  
      && a[k] <= \result.snd()) ) &&  
@      (a.length == 0 ==> \result.fst()==0 && \result.snd()==0);  
@*/
```

```
public static IntPair minMaxarray(int[] a, int i, int j) {  
    if (a.length == 0) {  
        IntPair res = new IntPair(0,0);  
        return res;  
    }  
    else {  
        IntPair res_sub = new IntPair(a[i],a[i]);  
        int k = i;  
        while (k<=j) {  
            if (a[k]<res_sub.fst()) {  
                res_sub.changeFst(a[k]);  
            }  
            if (a[k]>res_sub.snd()) {  
                res_sub.changeSnd(a[k]);  
            }  
            k++;  
        }  
        return res_sub;  
    }  
}
```



Una especificación formal (JML)
es usualmente chequeable

Violaciones de Aserciones

- La violación en tiempo de ejecución de una aserción de corrección es la manifestación de un defecto en el software.
- De acuerdo a los derechos y obligaciones enunciados anteriormente, la violación de una pre-condición es un error del cliente, mientras que la violación de una post-condición es un error del proveedor.
- Esto permite identificar responsables de defectos reflejados como violaciones a contratos.

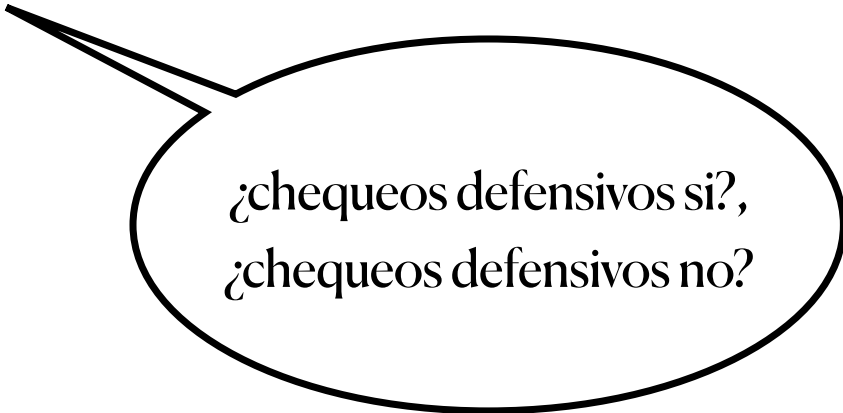
Formal vs Informal

```
public class IntegerSet {  
    ...  
  
    byte[] a; /* The array a is sorted */  
  
}
```

```
public class IntegerSet {  
    ...  
  
    byte[] a;  
  
    /*@ invariant  
    (\forall int i; 0 <= i && i < a.length-1; a[i] < a[i+1]);  
    @*/  
  
}
```

Especificaciones

```
public static IntPair minMaxArray(int[] a, int i, int j) {  
    if(a== null || i<0 || j<i || a.length <= j) {  
        throw new IllegalArgumentException("parámetros incorrectos");  
    }  
    ...  
}
```



¿chequeos defensivos si?,
¿chequeos defensivos no?

Los chequeos defensivos a veces no son posibles de ejecutar eficientemente. Ej. un método de búsqueda binaria requiere que su argumento este ordenado. Verificarlo requiere tiempo lineal, pero se supone que la búsqueda binaria es logarítmica!

Especificaciones

```
public static IntPair minMaxArray(int[] a, int i, int j) {  
    if(a== null || i<0 || j<i || a.length <= j) {  
        throw new IllegalArgumentException("parámetros incorrectos");  
    }  
    ...  
}
```

Las
especificaciones runtime
se deshabilitan antes del
release!

¿chequeos defensivos si?,
¿chequeos defensivos no?

Los chequeos defensivos a veces no son posibles de ejecutar eficientemente. Ej. un método de búsqueda binaria requiere que su argumento este ordenado. Verificarlo requiere tiempo lineal, pero se supone que la búsqueda binaria es logarítmica!

Invariantes de Clase

Invariantes de Clase

- Las precondiciones y postcondiciones establecen propiedades de rutinas particulares.
- Existe en general la necesidad de expresar otras propiedades globales de las instancias de una clase.

Invariantes de Clase

- Las precondiciones y postcondiciones establecen propiedades de rutinas particulares.
- Existe en general la necesidad de expresar otras propiedades globales de las instancias de una clase.

Invariantes de clase o representación

Invariantes de Clase

Invariantes de Clase

Todos los constructores deben establecer, al finalizar, la propiedad

Invariantes de Clase

Todos los constructores deben establecer, al finalizar, la propiedad

Todas las rutinas de la clase deben **preservar** la propiedad

Los Métodos y el Invariante de Representación

- Una clase respeta su contrato si y sólo si:
 - Los constructores de la clase garantizan la validez del invariante de representación en su terminación.
 - Cada método m satisface que:

$$\begin{array}{c} \{\text{pre-}m \text{ and } \text{Inv}\} \\ m \\ \{\text{post-}m \text{ and } \text{Inv}\} \end{array}$$

Ejemplo

```
/*@ invariant (1<=month && month<=12) &&
@ (day >= 1 && day<=31) &&
@ ((month==4 || month==6 || month==9 || month==11) => day<=30) &&
@ ((month=2 && leap(year) => day <= 29) &&
@ (( month=2 && !leap(year) => day <= 28) &&
@ (1900 <= year)
*/
```

```
public class Date {
```

```
    private int day;
    private int month;
    private int year;
```

```
    ...
```

```
}
```

Ejemplo

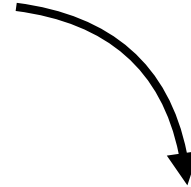
```
/*@ invariant (1<=month && month<=12) &&  
@ (day >= 1 && day<=31) &&  
@ ((month==4 || month==6 || month==9 || month==11) => day<=30) &&  
@ ((month=2 && leap(year) => day <= 29) &&  
@ (( month=2 && !leap(year) => day <= 28) &&  
@ (1900 <= year)  
*/
```

```
public class Date {
```

```
    private int day;  
    private int month;  
    private int year;
```

```
    ...
```

```
}
```



Ejemplo

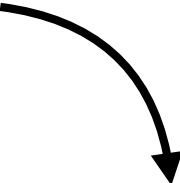
```
/*@ invariant (1<=month && month<=12) &&
@ (day >= 1 && day<=31) &&
@ ((month==4 || month==6 || month==9 || month==11) => day<=30) &&
@ ((month=2 && leap(year) => day <= 29) &&
@ (( month=2 && !leap(year) => day <= 28) &&
@ (1900 <= year)
*/
```

```
public class Date {
```

```
    private int day;
    private int month;
    private int year;
```

```
    ...
```

```
}
```



```
public boolean repOk() {
    if (month<1 || month>12)
        return false;
    if (day<1 || day>31)
        return false;

    ...
}
```

El proceso de V&V

- V&V debería aplicarse en cada instancia del proceso de desarrollo.
- Tiene dos objetivos principales:
 - descubrir defectos en el sistema, y
 - medir si el sistema es “usable” en situaciones de operación del mismo.
- Una forma de realizar tareas de V&V es a través de **análisis** (de programas, modelos, especificaciones, documentos, etc.).
 - En particular para código, tenemos **análisis estático** y **análisis dinámico**.

Análisis Dinámico

- Analiza las propiedades de programas mediante su ejecución:
 - Extrae información de las corridas de programas.

Análisis Dinámico

- Analiza las propiedades de programas mediante su ejecución:
 - Extrae información de las corridas de programas.



Preciso

Análisis Dinámico

- Analiza las propiedades de programas mediante su ejecución:
 - Extrae información de las corridas de programas.



Preciso



Incompleto

Análisis Dinámico

Análisis Dinámico

✓ Profiling de consumo de recursos (e.g. memoria)

Análisis Dinámico

✓ Profiling de consumo de recursos (e.g. memoria)

✓ Testing

Análisis Dinámico

✓ Profiling de consumo de recursos (e.g. memoria)

✓ Testing

✓ Chequeo en tiempo de ejecución de propiedades de programas.

Análisis Estático

- Se infieren propiedades de programas mediante el análisis del texto del mismo:
 - generalmente es impreciso

Análisis Estático

Análisis Estático

✓ Inspección Manual

Análisis Estático

✓ Inspección Manual

✓ Chequeo de tipos (en compilación)

Análisis Estático

✓ Inspección Manual

✓ Chequeo de tipos (en compilación)

✓ verificación estática de programas

Inspección manual

- Involucra personas que examinan el código fuente, para intentar descubrir defectos, y otros problemas de calidad “internos”
- No requiere la ejecución del sistema
- Técnica muy efectiva para descubrir defectos, aunque difícilmente exhaustiva.

Testing

- Esencialmente, consiste en comprobar el comportamiento de los programas en un conjunto de situaciones particulares (e.g., para algunas entradas particulares).
 - “Puede revelar la presencia de errores pero rara vez puede garantizar su ausencia.” *Dijkstra*
- Es la técnica de verificación funcional por excelencia.
- Debe usarse en combinación con otras técnicas de V&V estáticas (e.g., inspección de código) para dar mejores resultados.
- **Debe realizarse de manera planificada.**

Fundamentos de Testing

Edsger Wybe Dijkstra
(1930-2002)



Fundamentos de Testing

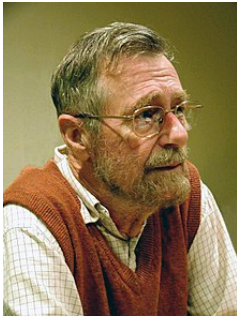
Edsger Wybe Dijkstra
(1930-2002)



Program testing can be used to show the presence of bugs, but never to show their absence!

E. W. Dijkstra. Notes On Structured Programming (EWD249), Section 3, 1970

Bonus track: Citas de Dijkstra



Bonus track: Citas de Dijkstra

The competent programmer is fully aware of the strictly limited size of his own skull; therefore he approaches the programming task in full humility, and among other things he avoids clever tricks like the plague. 1972.



Bonus track: Citas de Dijkstra

The competent programmer is fully aware of the strictly limited size of his own skull; therefore he approaches the programming task in full humility, and among other things he avoids clever tricks like the plague. 1972.

Don't blame me for the fact that competent programming, as I view it as an intellectual possibility, will be too difficult for "the average programmer" — you must not fall into the trap of rejecting a surgical technique because it is beyond the capabilities of the barber in his shop around the corner. 1975.



Bonus track: Citas de Dijkstra

The competent programmer is fully aware of the strictly limited size of his own skull; therefore he approaches the programming task in full humility, and among other things he avoids clever tricks like the plague. 1972.

Don't blame me for the fact that competent programming, as I view it as an intellectual possibility, will be too difficult for "the average programmer" — you must not fall into the trap of rejecting a surgical technique because it is beyond the capabilities of the barber in his shop around the corner. 1975.



Write a paper promising salvation, make it a 'structured' something or a 'virtual' something, or 'abstract', 'distributed' or 'higher-order' or 'applicative' and you can almost be certain of having started a new cult. 1979.

Bonus track: Citas de Dijkstra

The competent programmer is fully aware of the strictly limited size of his own skull; therefore he approaches the programming task in full humility, and among other things he avoids clever tricks like the plague. 1972.

Don't blame me for the fact that competent programming, as I view it as an intellectual possibility, will be too difficult for "the average programmer" — you must not fall into the trap of rejecting a surgical technique because it is beyond the capabilities of the barber in his shop around the corner. 1975.



Write a paper promising salvation, make it a 'structured' something or a 'virtual' something, or 'abstract', 'distributed' or 'higher-order' or 'applicative' and you can almost be certain of having started a new cult. 1979.

FORTRAN's tragic fate has been its wide acceptance, mentally chaining thousands and thousands of programmers to our past mistakes. 1972.

Bonus track: Citas de Dijkstra

The competent programmer is fully aware of the strictly limited size of his own skull; therefore he approaches the programming task in full humility, and among other things he avoids clever tricks like the plague. 1972.

If you want more effective programmers, you will discover that they should not waste their time debugging, they should not introduce the bugs to start with. 1972.



of rejecting a surgical technique because it is beyond the capabilities of the barber in his shop around the corner. 1975.

competent
Intellectual
the average
into the trap

Write a paper promising salvation, make it a 'structured' something or a 'virtual' something, or 'abstract', 'distributed' or 'higher-order' or 'applicative' and you can almost be certain of having started a new cult. 1979.

FORTTRAN's tragic fate has been its wide acceptance, mentally chaining thousands and thousands of programmers to our past mistakes. 1972.

Bonus track: Citas de Dijkstra

The competent programmer is fully aware of the strictly limited size of his own skull; therefore he approaches the programming task in full humility, and among other things he avoids clever tricks like the plague. 1972.

If you want more effective programmers, you will discover that they should not waste their time debugging, they should not introduce the bugs to start with. 1972.



of rejecting a surgical technique because it is beyond the capabilities of the barber in his shop around the corner. 1975.

competent
Intellectual
the average
into the trap

Write a paper promising
Besides a mathematical inclination, an exceptionally good mastery of one's native tongue is the most vital asset of a competent programmer. 1975.

higher order of
'applicative' and you can almost be certain of having started a new cult. 1979.
FORTRAN's tragic fate has been its wide acceptance, mentally chaining thousands and thousands of programmers to our past mistakes. 1972.

Bonus track: Citas de Dijkstra

The competent programmer is fully aware of the strictly limited size of his own skull; therefore he approaches the programming task in full humility, and among other things he avoids clever tricks like the plague. 1972.

If you want more effective programmers, you will discover that they should not waste their time debugging, they should not introduce the bugs to start with. 1972.



The use of COBOL cripples the mind; its teaching should, therefore, be regarded as a criminal offense. 1975.

Write a paper promising
Besides a mathematical inclination, an exceptionally good mastery of one's native tongue is the most vital asset of a competent programmer. 1975.

higher order of
'applicative' and you can almost be certain of having started a new cult. 1979.

FORTRAN's tragic fate has been its wide acceptance, mentally chaining thousands and thousands of programmers to our past mistakes. 1972.

Bonus track: Citas de Dijkstra

The competent programmer is fully aware of the strictly limited size of his own skull, therefore he approaches the programming task in full humility, and among other things he avoids clever tricks like the plague. 1972.

If you want more effective programmers, you will discover that they should not waste their time debugging, they should not introduce the bugs to start with. 1972.



The use of COBOL cripples the mind; its teaching should, therefore, be regarded as a criminal offense. 1975.

Write a paper promising: Besides a mathematical inclination, an exceptionally good mastery of one's native tongue is the most vital asset of a competent programmer. 1975.

higher order of 'applicative' and you can almost be certain of having started a new cult. 1979.

FORTRAN's tragic fate has been its wide acceptance, mentally chaining thousands and thousands of programmers to our past mistakes. 1972.

Bonus track: Citas de Dijkstra

The simplest programming is full awareness of
Simplicity is prerequisite for reliability. 1975.
the strictly limited size of his own skull,
therefore he approaches the programming task
in full humility, and among other things
avoids clever tricks like the plague. 197

If you want more effective programming
will discover that they should not waste
time debugging, they should not introduce
bugs to start with. 1972.



The use of COBOL cripples the mind; its teaching
should, therefore, be regarded as a criminal
offense. 1975.

Programming is one of the most
difficult branches of applied
mathematics; the poorer
mathematicians had better remain
pure mathematicians. 1975.

Write a paper promising
Besides a mathematical inclination,
an exceptionally good mastery of
one's native tongue is the most
competent
5.

've' and you can
certain of having
started a new cult. 1979.
FORTRAN's tragic fate
has been its wide
acceptance, mentally
chaining thousands and
thousands of
programmers to our past
mistakes. 1972.

Bonus track: Citas de Dijkstra

The simplest programming is fully aware of the strictly limited size of his own skull, therefore he approaches the programming task in full humility, and among other things avoids clever tricks like the plague. 1975.

If you want more effective programming, you will discover that they should not waste time debugging, they should not introduce bugs to start with. 1972.



The use of COBOL cripples the mind; its teaching should, therefore, be regarded as a criminal offense. 1975.

Programming is one of the most difficult branches of applied mathematics; the poorer mathematicians had better remain pure mathematicians. 1975.

Write a paper promising. Besides a mathematical inclination, an exceptionally good mastery of one's native tongue is the most

competent

1975.

'and you can be certain of having

started a new cult. 1979.

FORTRAN's tragic fate

How do we convince people that in programming simplicity and clarity—in short: what mathematicians call "elegance"—are not a dispensable luxury, but a crucial matter that decides between success and failure? 1982.

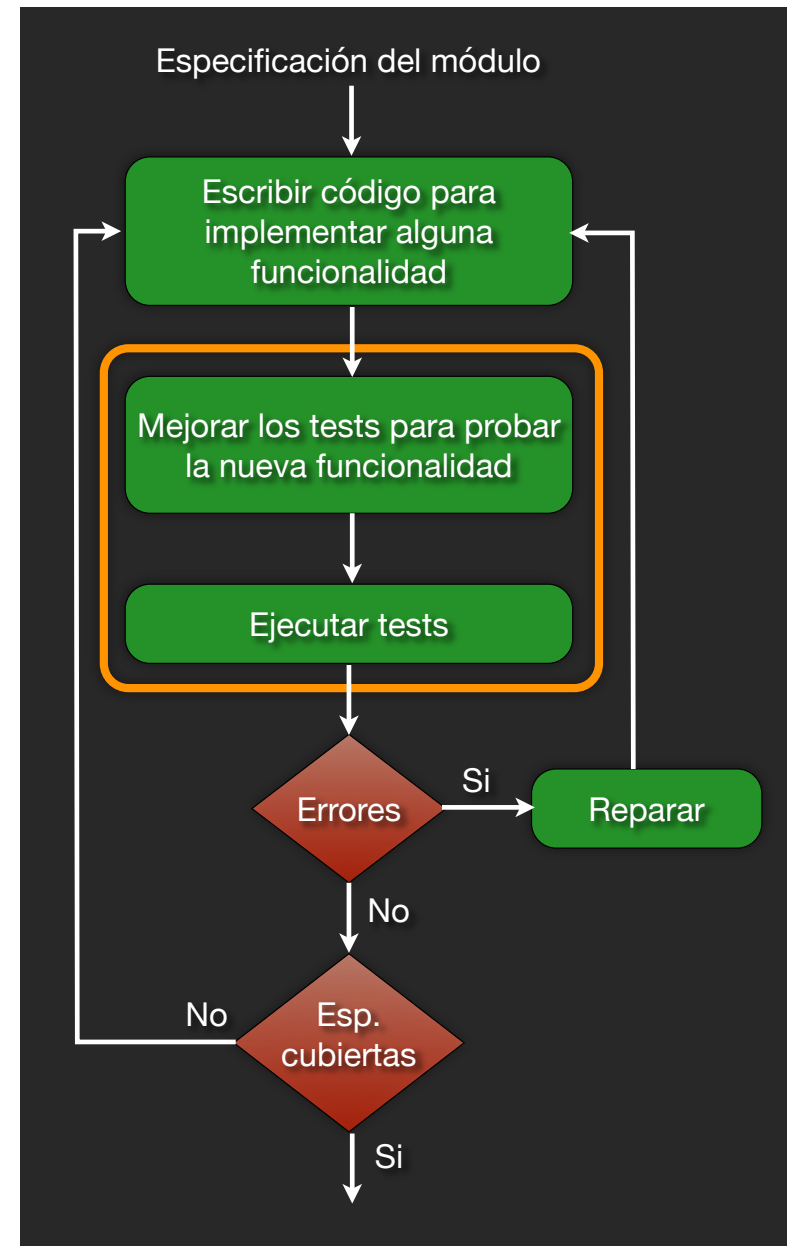
Testing en procesos de desarrollo clásicos

- Los modelos de proceso de desarrollo clásicos involucran generalmente las siguientes actividades:
 - Fase de requisitos
 - Fase de especificación
 - Fase de planeamiento
 - Fase de diseño
 - Fase de implementación
 - Integración y Testing
 - Mantenimiento
- Testing es una actividad realizada en este caso posteriormente a la implementación.

Testing en procesos de desarrollo ágiles

En los modelos de proceso de desarrollo “ágiles”, el testing está presente en varias fases del proceso de desarrollo, principalmente como parte de la codificación.

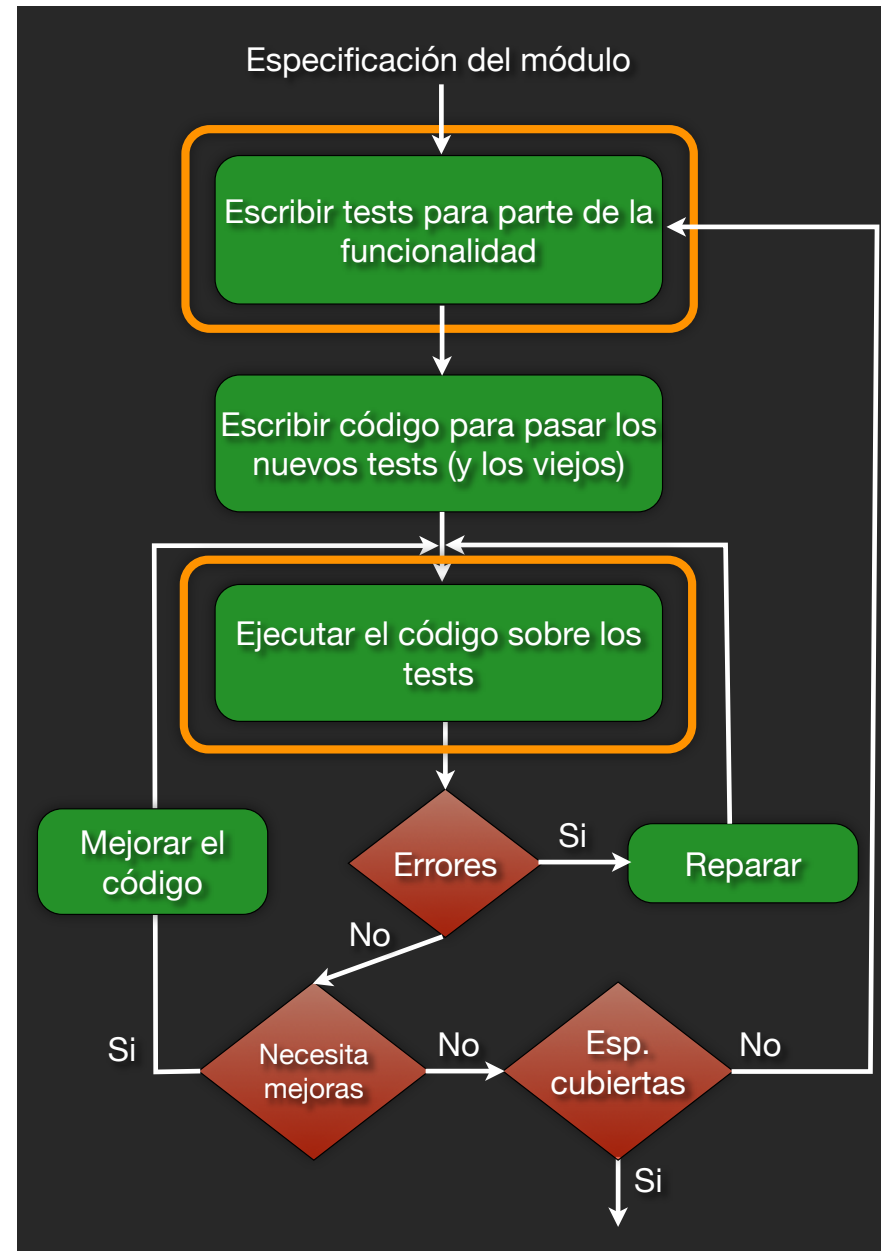
En el desarrollo de software guiado por funcionalidades, el testing se realiza como parte de la implementación de cada funcionalidad.



Testing como especificaciones

Algunos procesos “ágiles” ponen un énfasis aún mayor en testing.

En Test Driven Development (TDD), los tests se escriben antes de la codificación, y sirven como especificación de funcionalidades, y criterio de aceptación de código



Testing de software

Tenemos, en principio, una versión del **testing** como un **proceso destructivo** consistente en **encontrar** errores (cuya presencia es asumida) en un programa.

La aplicación de un **test** es **exitosa** si hace que el **programa falle** (e.g. devuelva un valor inesperado, produzca una excepción no prevista, etc).

Esta definición puede ser matizada y mejorada si consideramos una definición del término basada en **nivel de pensamiento**.

Definición de Testing por Niveles

Definición de Testing por Niveles

Nivel 0: No hay diferencia entre testing y debugging.

Definición de Testing por Niveles

Nivel 0: No hay diferencia entre testing y debugging.

Nivel 1: El propósito de testing es mostrar corrección.

Definición de Testing por Niveles

Nivel 0: No hay diferencia entre testing y debugging.

Nivel 1: El propósito de testing es mostrar corrección.

Nivel 2: El propósito de testing es mostrar que el software no funciona.

Definición de Testing por Niveles

Nivel 0: No hay diferencia entre **testing** y **debugging**.

Nivel 1: El propósito de testing es mostrar **corrección**.

Nivel 2: El propósito de testing es mostrar que el **software no funciona**.

Nivel 3: El propósito de testing no es probar nada específico, sino **reducir los riesgos** asociados con la utilización del software.

Definición de Testing por Niveles

Nivel 0: No hay diferencia entre **testing** y **debugging**.

Nivel 1: El propósito de testing es mostrar **corrección**.

Nivel 2: El propósito de testing es mostrar que el **software no funciona**.

Nivel 3: El propósito de testing no es probar nada específico, sino **reducir los riesgos** asociados con la utilización del software.

Nivel 4: Testing es una **disciplina mental** que ayuda a que los profesionales desarrollen software con más calidad.

Nivel 3 de pensamiento

El propósito de testing no es probar nada específico, sino **reducir los riesgos asociados con la utilización del software.**

El testing solo puede mostrar la **presencia de fallos**.

Siempre que usamos un software debemos asumir un cierto **riesgo**.

Este riesgo puede ser **pequeño** y sus consecuencias irrelevantes.

Sin embargo, este riesgo también puede ser grande y tener consecuencias catastróficas.

Testeadores y desarrolladores deben cooperar para **reducir riesgos**.

Nivel 4 de pensamiento

Testing es una **disciplina mental** que ayuda a que los profesionales desarrollen software con más calidad.

El testing es solo un método para aumentar la calidad.

La principal responsabilidad es **medir y mejorar** la calidad del software.

Nociones a tener en cuenta a la hora de testear

- ✓ Una parte necesaria a la hora de definir un test consiste en proporcionar el resultado esperado.
- ✓ Por tanto, cada test debe tener dos componentes (más adelante veremos que es un poco más complejo):
 - ✓ Una enumeración de los valores de los **inputs** del programa.
 - ✓ Una definición precisa del **output correcto** para esos valores de los inputs.

Nociones a tener en cuenta a la hora de testear

- ✓ Una parte necesaria a la hora de definir un test consiste en proporcionar el resultado esperado.
- ✓ Por tanto, cada test debe tener dos componentes (más adelante veremos que es un poco más complejo):
 - ✓ Una enumeración de los valores de los **inputs** del programa.
 - ✓ Una definición precisa del **output correcto** para esos valores de los inputs.
- ✓ Un programador debería evitar testear sus propios programas, no estamos preparados (psicológicamente) para buscar errores en nuestro propio trabajo.
- ✓ En particular, el programa puede tener errores debido a que el programador no ha comprendido el enunciado del problema o su especificación.

Nociones a tener en cuenta a la hora de testear

¡Warning!! Esto está cambiando con la aparición de las metodologías ágiles.

- ✓ Una parte necesaria a la hora de definir un test consiste en proporcionar el resultado esperado.
- ✓ Por tanto, cada test debe tener dos componentes (más adelante veremos que es un poco más complejo):
 - ✓ Una enumeración de los valores de los **inputs** del programa.
 - ✓ Una definición precisa del **output correcto** para esos valores de los inputs.
- ✓ Un programador debería evitar testear sus propios programas, no estamos preparados (psicológicamente) para buscar errores en nuestro propio trabajo.
- ✓ En particular, el programa puede tener errores debido a que el programador no ha comprendido el enunciado del problema o su especificación.

Nociones a tener en cuenta a la hora de testear

- ✓ Testear un programa para ver si no hace lo que se supone que debe hacer es solo la mitad del proceso; la otra mitad consiste en ver si el programa hace algo que no debería hacer.
- ✓ Evita utilizar tests desechables.

Nociones a tener en cuenta a la hora de testear

- ✓ Testear un programa para ver si no hace lo que se supone que debe hacer es solo la mitad del proceso; la otra mitad consiste en ver si el programa hace algo que no debería hacer.
- ✓ Evita utilizar tests desechables.
 - ✓ Si tenemos que testear el programa de nuevo entonces tenemos que reinventar los tests.
- ✓ Guardar tests y ejecutarlos tras cambios en otras componentes del programa se conoce como **regression testing**.

Testing: Terminología Básica

Defecto (Fault): Línea(s) de código defectuosas en el software

Error: Un estado interno incorrecto que es la manifestación de algún defecto.

Falla (failure): comportamiento incorrecto externo con respecto a los requerimientos o alguna otra descripción del comportamiento esperado.

Faults, errors & failures: Ejemplo

- ✓ Un paciente le enumera al médico un lista de **síntomas**.
 - ✓ Failures.
- ✓ El médico intenta diagnosticar la causa de la dolencia.
 - ✓ Fault (defecto)
- ✓ El médico puede buscar **condiciones anómalas internas** (presión arterial alta, arritmias, bacterias en el torrente sanguíneo).
 - ✓ Errors.

Faults, errors & failures: Ejemplo

- ✓ Un paciente le enumera al médico un lista de **síntomas**.
 - ✓ Failures.
- ✓ El médico intenta diagnosticar la causa de la dolencia.
 - ✓ Fault (defecto)
- ✓ El médico puede buscar **condiciones anómalas internas** (presión arterial alta, arritmias, bacterias en el torrente sanguíneo).
 - ✓ Errors.

La mayoría de los problemas médicos provienen o de ataques externos (bacterias, virus) o de la degradación natural debida al envejecimiento.

Faults, errors & failures: Ejemplo

- ✓ Un paciente le enumera al médico un lista de **síntomas**.
 - ✓ Failures.
- ✓ El médico intenta diagnosticar la causa de la dolencia.
 - ✓ Fault (defecto)
- ✓ El médico puede buscar **condiciones anómalas internas** (presión arterial alta, arritmias, bacterias en el torrente sanguíneo).
 - ✓ Errors.

La mayoría de los problemas médicos provienen o de ataques externos (bacterias, virus) o de la degradación natural debida al envejecimiento.

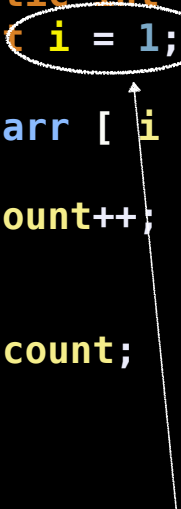
Software faults estaban allí desde el principio y no “aparecen” cuando una componente se “gasta”.

Un ejemplo concreto

```
/**
 *Este método calcula la cantidad de ceros que hay en el arreglo
 @precondition. a != null
 @postcondition. a no cambia. El resultado es la cantidad de ceros
 en el arreglo
 */
public static int numZero (int [ ] arr){ int count = 0;
    for (int i = 1; i < arr.length; i++)
    {
        if (arr [ i ] == 0)
        {
            count++;
        }
    }
    return count;
}
```

Un ejemplo concreto

```
/**
 *Este método calcula la cantidad de ceros que hay en el arreglo
 @precondition. a != null
 @postcondition. a no cambia. El resultado es la cantidad de ceros
 en el arreglo
 */
public static int numZero (int [ ] arr){ int count = 0;
    for (int i = 1; i < arr.length; i++)
    {
        if (arr [ i ] == 0)
        {
            count++;
        }
    }
    return count;
}
```



Defecto: Debería comenzar la búsqueda en 0, no 1

Un ejemplo concreto

```
/**
 *Este método calcula la cantidad de ceros en el arreglo
 *@precondition. a != null
 *@postcondition. a no cambia. El arreglo no cambia.
 */
public static int numZero (int[] a) {
    for (int i = 1; i < a.length; i++) {
        if (a[i] == 0) {
            count++;
        }
    }
    return count;
}
```

```
@Test
public void test() {
    int[] a = {2,7,0};
    int p = SimpleRoutines.numZero(a);
    assertTrue(p==1);
} //valor esperado
Valor Obtenido: 1
Error: i es 1, no 0, en la primera iteración
Falla: ninguna
```

Defecto: Debería comenzar la búsqueda en 0, no 1

Un ejemplo concreto

```
/**
 *Este método calcula la cantidad de ceros en el arreglo
 *precondition. a != null
 *postcondition. a no cambia. El arreglo no cambia.
 */
public static int numZero (int[] a) {
    for (int i = 1; i < a.length; i++) {
        if (a[i] == 0) {
            count++;
        }
    }
    return count;
}
```

Defecto: Debería comenzar la búsqueda en 0, no 1

```
@Test
public void test() {
    int[] a = {2,7,0};
    int p = SimpleRoutines.numZero(a);
    assertTrue(p==1);
} //valor esperado
Valor Obtenido: 1
Error: i es 1, no 0, en la primera iteración
Falla: ninguna
```

```
@Test
public void test() {
    int[] a = {0,2,7};
    int p = SimpleRoutines.numZero(a);
    assertTrue(p==1);} //valor esperado
Valor Obtenido: 0
Error: i es 1, no 0, en la primera iteración
Falla: count es 0 al retornar
```

Más definiciones

- **Testing**: Evaluar el software observando su ejecución
- **Debugging**: El proceso de buscar un defecto dada una falla
- Un test es una prueba de software, compuesta usualmente de:
 - Entradas
 - Código a ejecutar (testear)
 - Descripción de los resultados esperados (condición de aceptación)
- **Test exitoso**: Ejecución de un test no revela una falla
- **Test fallido**: Ejecución de un test descubre una falla

Más definiciones

- **Testing**: Evaluar el software observando su ejecución
 - **Debugging**: El proceso de buscar un defecto dada una falla
 - Un test es una prueba de software, compuesta usualmente de:
 - Entradas
 - Código a ejecutar (testear)
 - Descripción de los resultados esperados (condición de aceptación)
 - **Test exitoso**: Ejecución de un test no revela una falla
 - **Test fallido**: Ejecución de un test descubre una falla
- procesos distintos!!

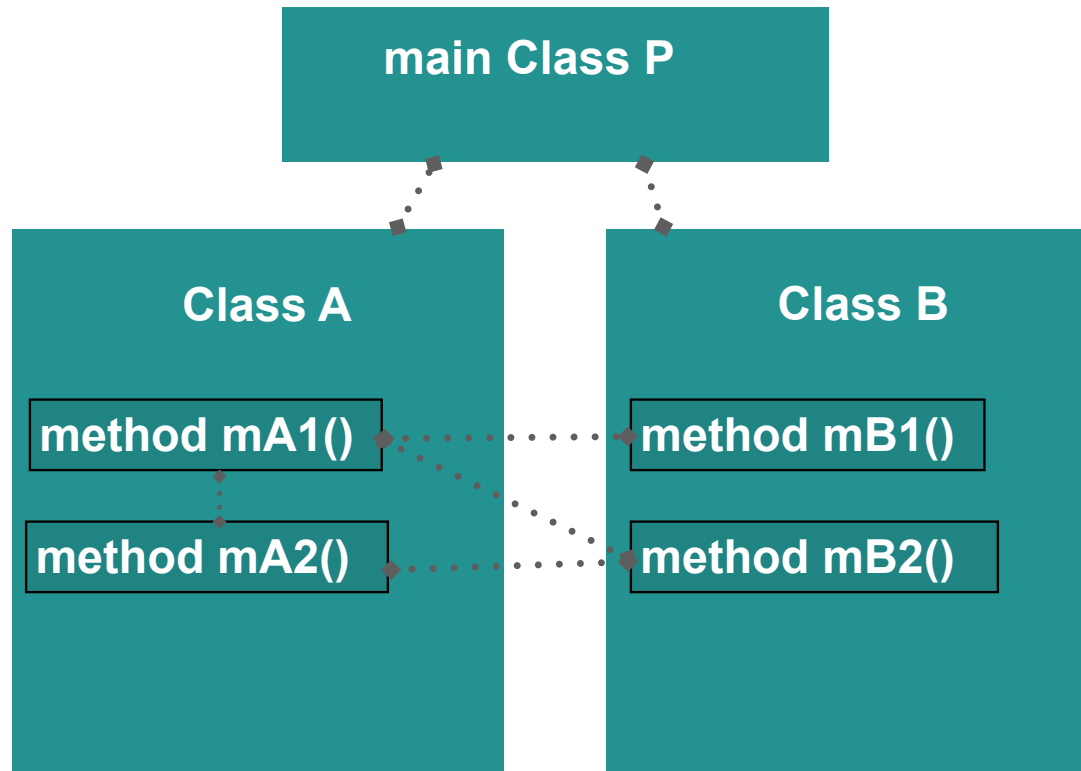
Testing: clasificaciones básicas

- Existen diferentes tipos de testing, de acuerdo a las características de sus partes. algunos de estos tipos son los siguientes:
 - **Regresión**: la condición de aceptación es preservar el comportamiento de versiones anteriores del software.
 - **Diferencial**: la condición de aceptación es mantener un comportamiento similar a otro software con el mismo propósito que el testeado.
 - **Aceptación**: Evalua el software con respecto a los requerimientos o necesidades del usuario

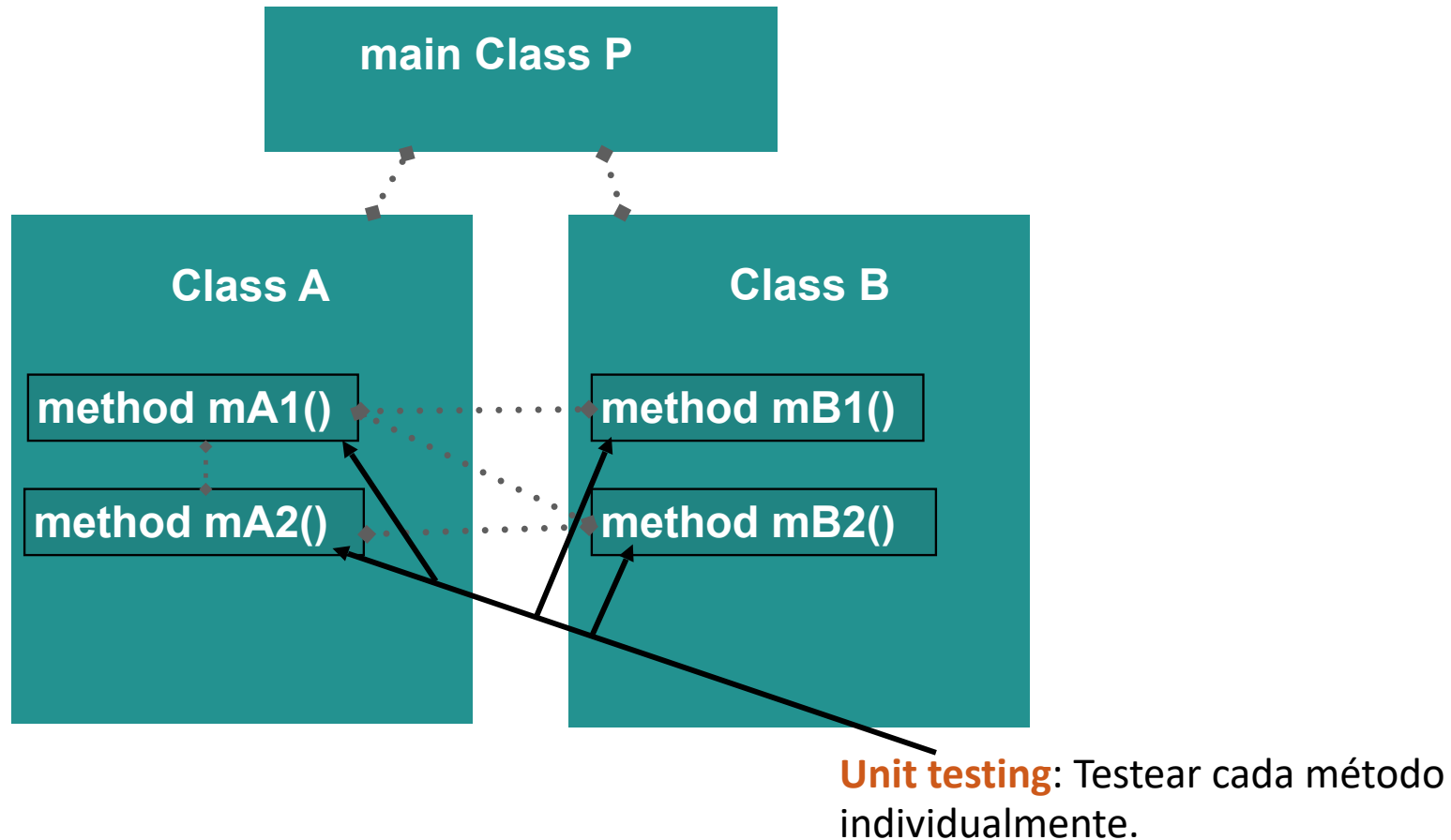
Testing de Aceptación

- Es el testing que se lleva adelante para determinar si un sistema cumple con su correspondiente “criterio de aceptación”, es decir, los requisitos establecidos para el mismo.
 - **Testing de Aceptación de Usuario:** lo lleva adelante el usuario, para determinar si cumple con sus necesidades/expectativas
 - validación
 - **Testing de Aceptación de Negocio:** lo lleva adelante el equipo de desarrollo/testing, para determinar si el sistema eventualmente pasará las pruebas de aceptación de usuario
 - verificación

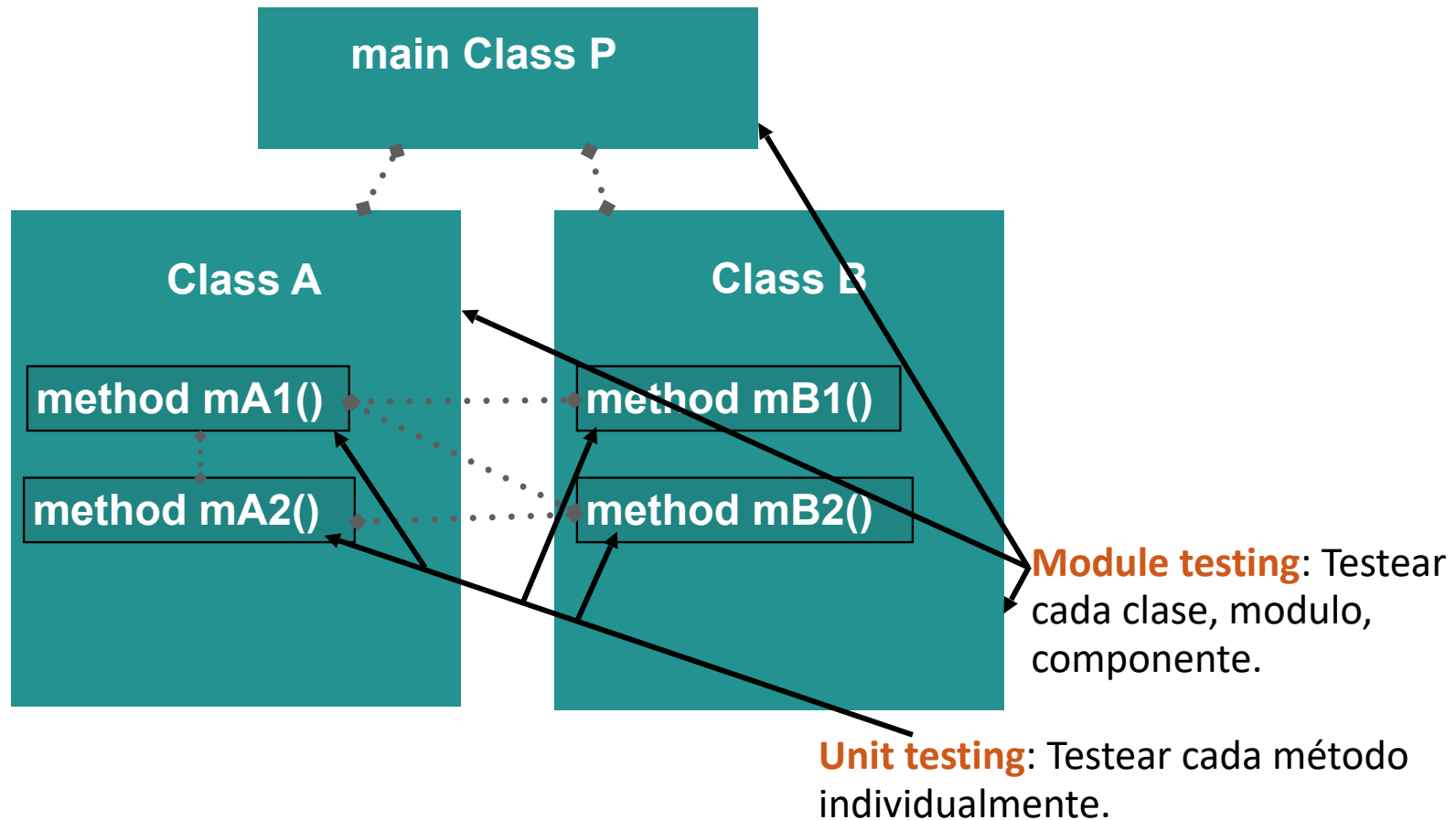
Niveles de Testing



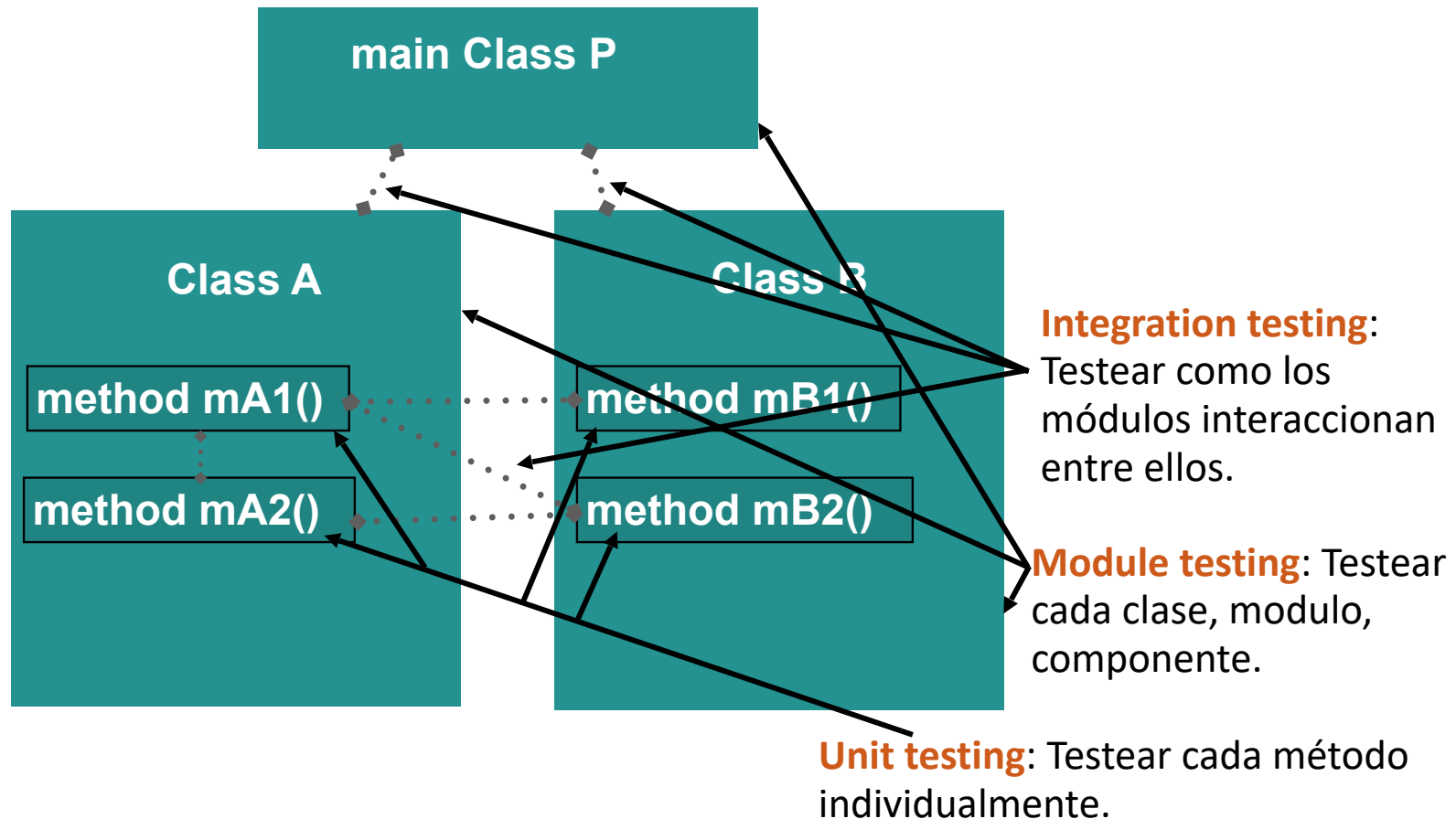
Niveles de Testing



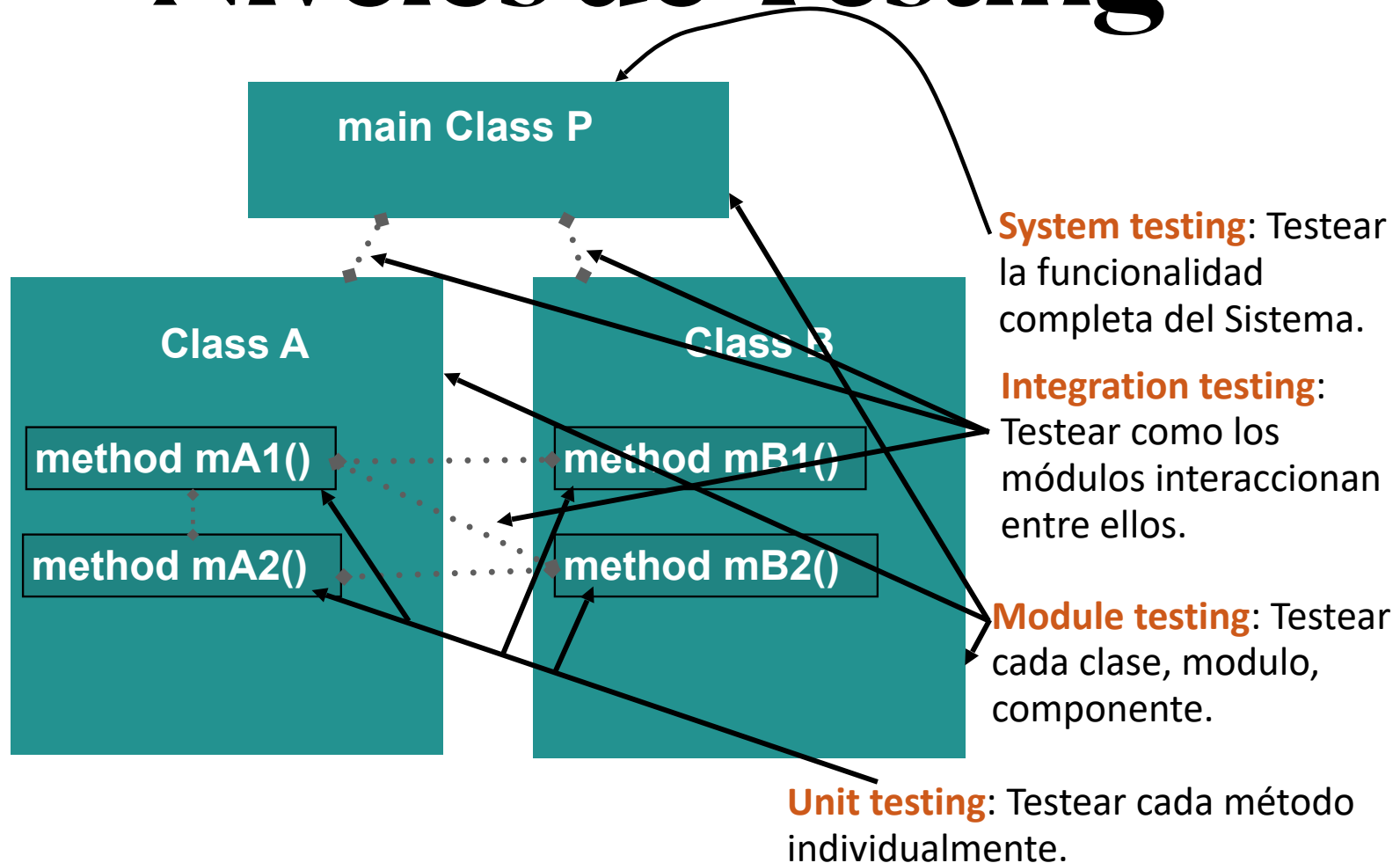
Niveles de Testing



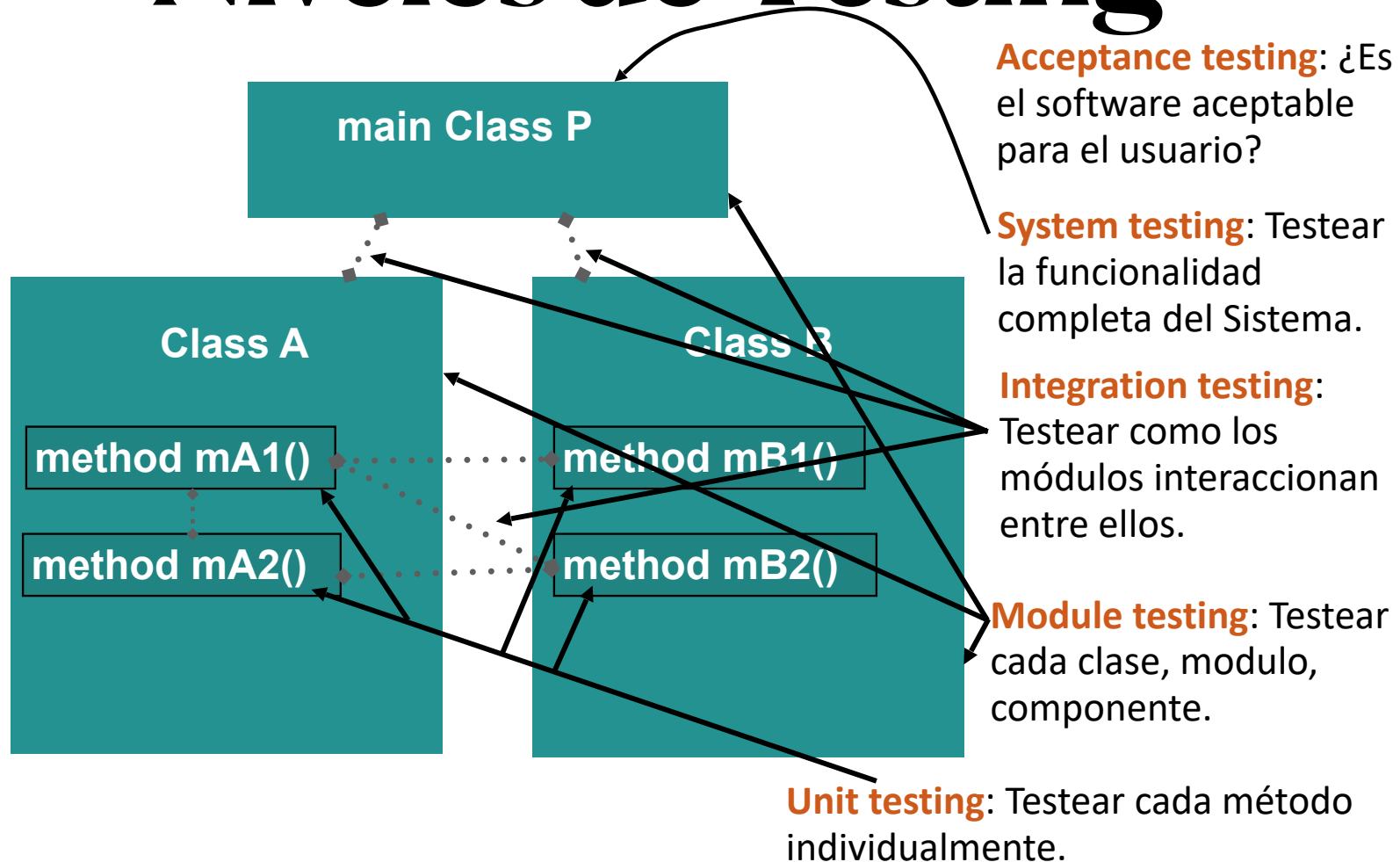
Niveles de Testing



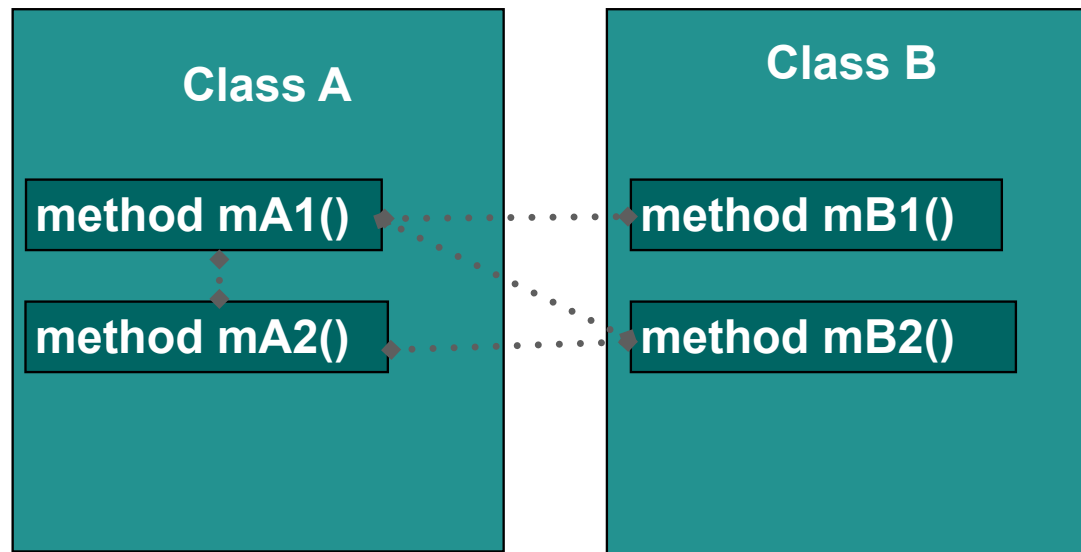
Niveles de Testing



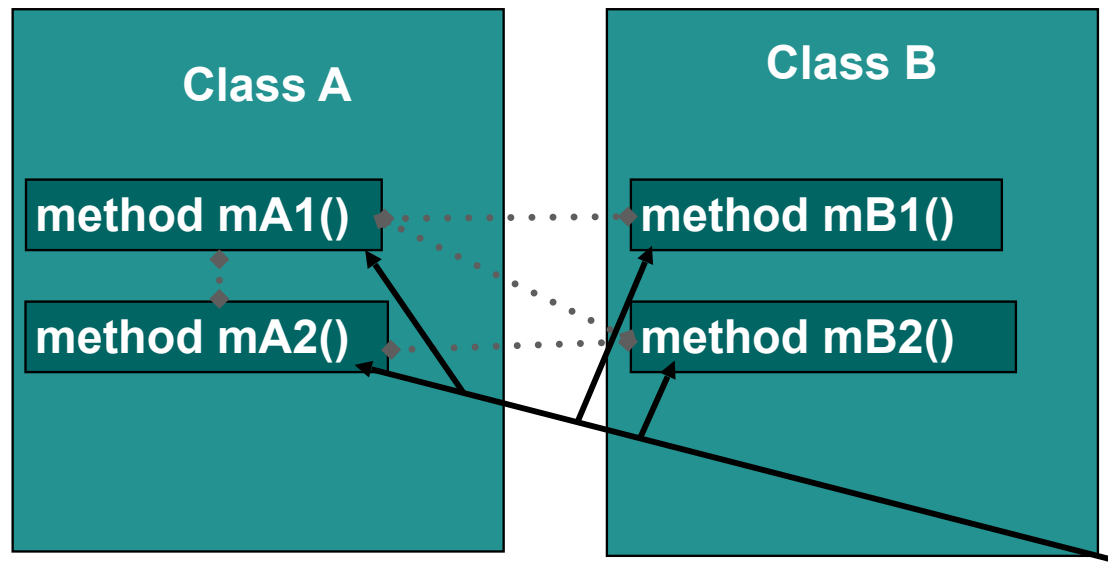
Niveles de Testing



Niveles de testing POO

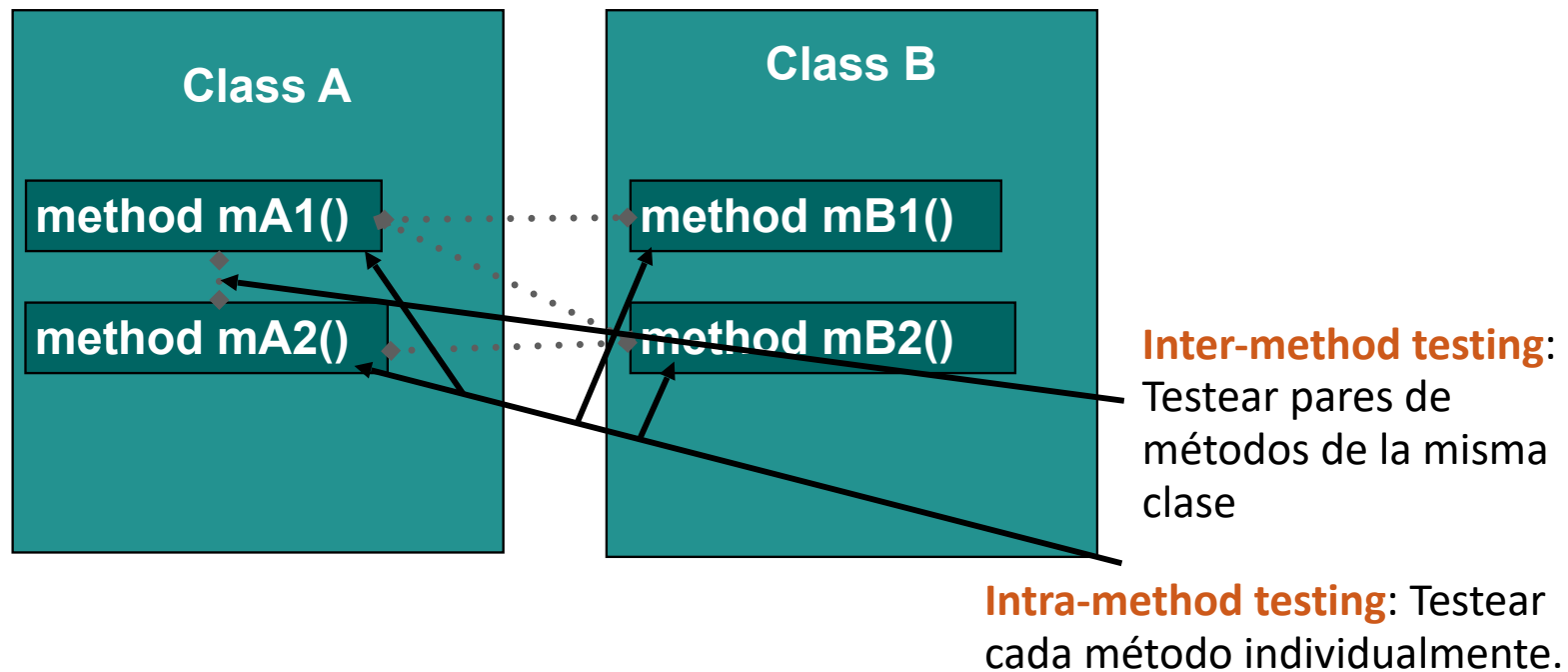


Niveles de testing POO

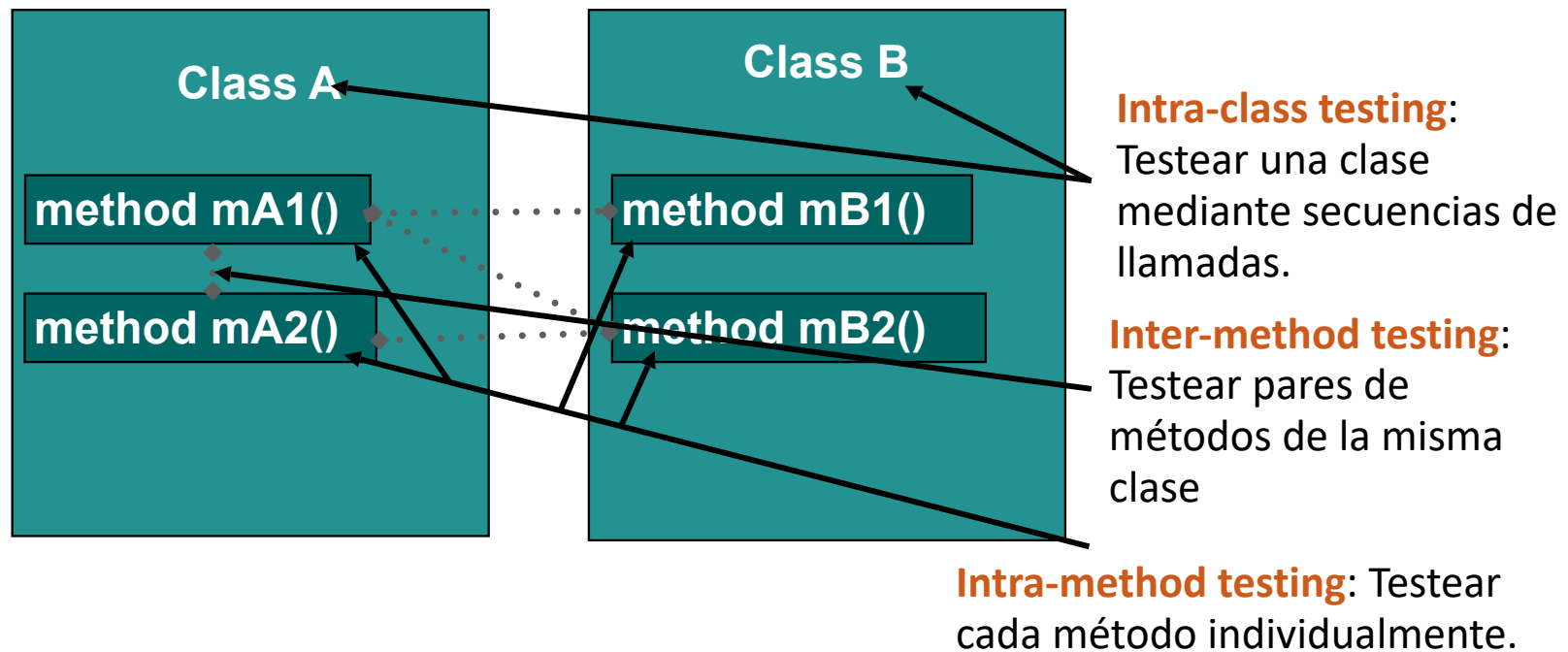


Intra-method testing: Testear cada método individualmente.

Niveles de testing POO



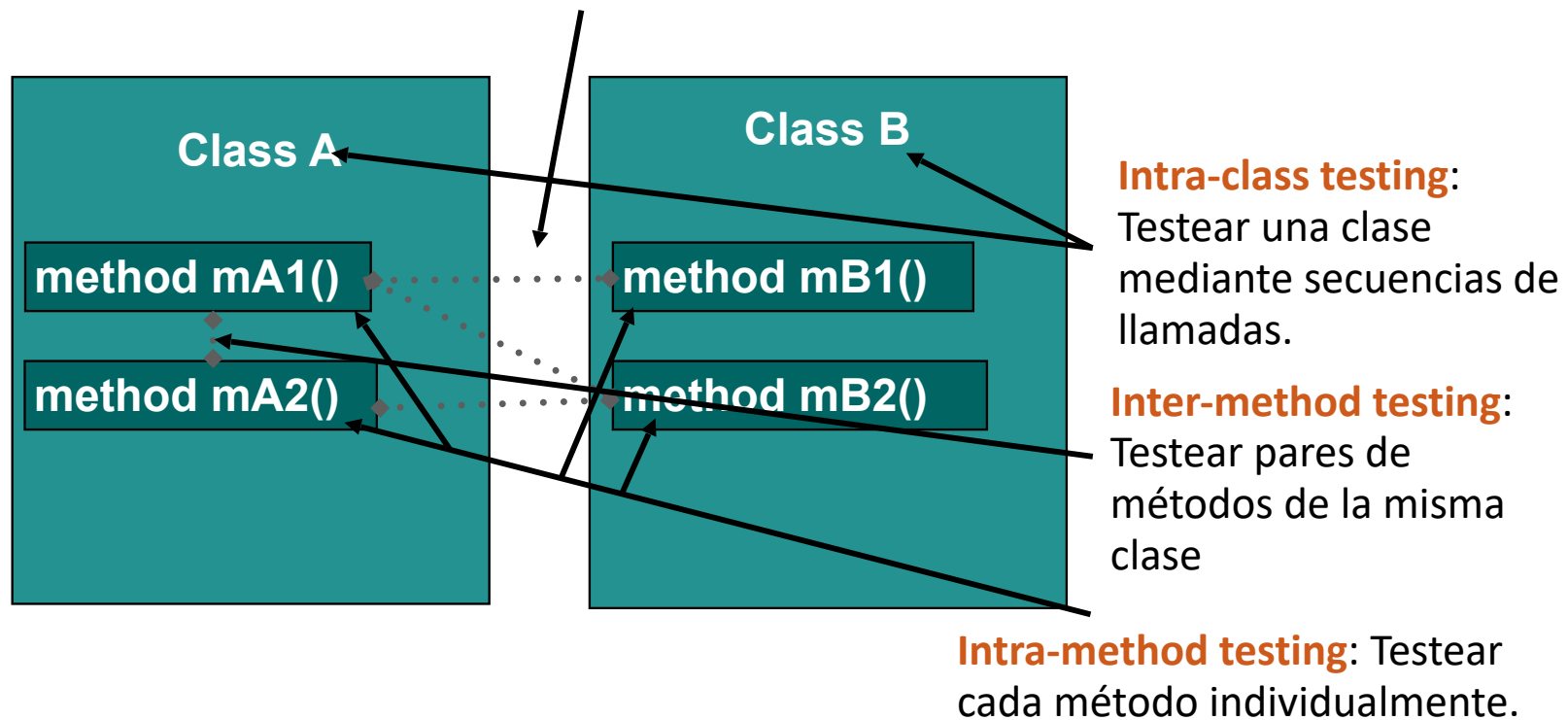
Niveles de testing POO



Niveles de testing POO

Inter-class testing:

Testear múltiples clases
juntas



Conclusión: ¿Por qué testeamos el software?

El objetivo del testing es revelar fallas tan temprano como sea posible durante el desarrollo, para:

- ✓ Mejorar la calidad del software
- ✓ Reducir el costo
- ✓ Mantener a los usuarios satisfechos

✓

Lectura Recomendada

Leer el capítulo 1 del libro "Introduction to Software Testing" de Ammann y Offutt